

P0468R1

An Intrusive Smart Pointer

Published Proposal, 2018-06-19

This version:

<https://wg21.link/p0468r1>

Author:

[Isabella Muerte](#)

Audience:

LEWG, LWG, SG1

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P0468](#)

Current Source:

[slurps-mad-rips/papers/proposals/retain_ptr](https://ericniebler.com/2018/06/19/slurps-mad-rips/papers/proposals/retain_ptr)

Abstract

A smart pointer is needed to manage objects with internal reference counts to reduce maintenance overhead of C and C++ APIs.

Table of Contents

1	Acknowledgements
2	Revision History
2.1	Revision 1
2.2	Revision 0
3	Motivation
4	Scope and Impact
5	Frequently Asked Questions
5.1	How does <code>intrusive_ptr</code> not meet the needs of modern C++?
5.2	Why is it called <code>retain_ptr</code> and not <code>intrusive_ptr</code> ?
5.3	Is <code>retain_ptr</code> atomic?
5.4	Does <code>retain_ptr</code> support allocators?
5.5	Can <code>retain_ptr</code> be constexpr?
5.6	Does <code>retain_ptr</code> adopt or retain an object on construction?
5.7	Why provide <code>retain_object_t</code> and <code>adopt_object_t</code> ?
5.8	Does <code>retain_ptr</code> overload the addressof operator?
5.9	Can <code>retain_traits</code> store state?
5.10	Why not just wrap a <code>unique_ptr</code> with a custom deleter?
6	Examples
6.1	OpenCL
6.2	ObjC Runtime
6.3	DirectX and COM

6.4

7 Technical Specification7.1 `atomic_reference_count<T>` and `reference_count<T>`7.2 `retain_object_t` and `adopt_object_t`7.3 `retain_traits<T>`7.4 `retain_ptr<T>`7.4.1 `retain_ptr` constructors7.4.2 `retain_ptr` destructor7.4.3 `retain_ptr` assignment7.4.4 `retain_ptr` observers7.4.5 `retain_ptr` modifiers7.4.6 `retain_ptr` specialized algorithms**References**

Informative References

§ 1. Acknowledgements

I'd like to thank the following:

- Jackie Kay and Brittany Friedman for encouraging me to submit this proposal.
- Gor Nishanov, whose coroutine slides at CppCon 2016 gave me the final push to sit down and write this.
- Bryce "Wash" Lebach for representing this paper when I was unable to.
- Mark Zeren for *constantly* bugging me to see if I might update this paper.
- JF Bastien and Billy O'Neal for assisting me with some atomic operations, which I am absolute garbage at.

§ 2. Revision History

§ 2.1. Revision 1

- Rewrote proposal in bikeshed, the hot new format that every cool proposal writer is using now.
- Clarified section on addressof operator, now that `out_ptr` is being submitted
- Removed `retain_ptr::unique` as it was removed from `shared_ptr` for C++20.
- Removed code example of implementing a `std::future`-like type.
- Changed `retain_ptr::detach` to `retain_ptr::release`
- Changed name of `retain_t` to `retain_object_t`
- Changed atomicity of `atomic_reference_count` based on suggestions from Billy O'Neal.
- Changed wording to use C++17/C++20 features.
- Changed order of document. [§7 Technical Specification](#) is now at the very end of the document, and [§6 Examples](#) are further up.
- Added `adopt_object_t` and `default_action` to let users decide what action they want on construction and reset.
- Added `static_pointer_cast`, `dynamic_pointer_cast`, `const_pointer_cast`, and `reinterpret_pointer_cast`.
- Added deduction guides to bring `retain_ptr` in line with other smart pointers
- Added more code examples

§ 2.2. Revision 0

Initial Release

§ 3. Motivation

There are a wide variety of C and C++ APIs that rely on reference counting, but either because of the language (C) or the age of the library (C++), they are unable to be safely use with either `std::unique_ptr<T>` or `std::shared_ptr<T>`. In the former case, there is no guaranteed way to make sure the `unique_ptr` is the last instance (i.e., that it is *unique*), and in the latter case, `shared_ptr` manages its own API. In addition, existing intrusive smart pointers such as `boost::intrusive_ptr<T>`, [Microsoft's ComPtr<T>](#), or [WebKit's aptly named WTF::RefPtr<T>](#) do not meet the needs of modern C++ smart pointers or APIs. This proposal attempts to solve these shortcomings in an extensible and future proof manner, while permitting a simple upgrade path for existing project specific intrusive smart pointers and providing the opportunity for value semantics when interfacing with opaque interfaces.

Those that work on systems or tools that rely on reference counting stand to benefit most from `retain_ptr`. Additionally, `retain_ptr` would be a benefit to standard library implementers for types that *secretly* use intrusive reference counting such as a coroutines capable future and promise.

If `retain_ptr` is added to the standard library, the C++ community would also be one step closer to a non-atomic `shared_ptr` and `weak_ptr`, much like Rust's `Rc<T>`.

A reference implementation of `retain_ptr`, along with examples of its use, can be found on [github](#).

§ 4. Scope and Impact

`retain_ptr<T, R>` would ideally be available in the `<memory>` standard header. It is a pure extension to the C++ standard library and can (more or less) be implemented using any conforming C++14 or C++11 compiler with very little effort. See the [§7 Technical Specification](#) for interface and behavior details.

§ 5. Frequently Asked Questions

Several common questions regarding the design of `retain_ptr<T, R>` can be found below.

§ 5.1. How does `intrusive_ptr` not meet the needs of modern C++?

`boost::intrusive_ptr` has had nearly the same interface since its introduction in 2001 by Peter Dimov. Furthermore, `boost::intrusive_ptr` has several failings in its API that cannot be changed from without breaking compatibility. When constructing a `boost::intrusive_ptr`, by default it increments the reference count. This is because of its `intrusive_ref_count` mixin, which starts with a reference count of 0 when it is default constructed. Out of all the libraries I tried to look at, this was the one instance where an object required it be incremented after construction. This proposal rectifies this by permitting each traits instance to choose its default action during construction.

Additionally, `boost::intrusive_ptr` does not have the ability to "overload" its `pointer` type member, requiring some additional work when interfacing with C APIs (e.g., `boost::intrusive_ptr<decltype(*declval<cl_mem>())>`). This also precludes it from working with types that meet the *NullablePointer* named requirements (a feature that `unique_ptr` supports).

Furthermore, `boost::intrusive_ptr` relies on ADL calls of two functions:

- `intrusive_add_ref`
- `intrusive_release`

While this approach is fine in most cases, it does remove the ability to easily "swap out" the actions taken when increment or decrementing the reference count (e.g., setting a debugger breakpoint on a specific traits implementation, but not on *all* traits implementations). This naming convention uses terms found

in Microsoft's COM. While this isn't an issue per se, it would be odd to have functions with those names found in the standard.

§ 5.2. Why is it called `retain_ptr` and not `intrusive_ptr`?

`retain_ptr` diverges from the design of `boost::intrusive_ptr`. It was decided to change the name so as to not cause assumptions of `retain_ptr`'s interface and behavior.

Some additional names that might be considered (for bikeshedding) are:

- `extend_ptr`
- `counted_ptr`
- `borrow_ptr`
- `mutual_ptr`
- `joint_ptr`

Comedy Option:

- `auto_ptr`

§ 5.3. Is `retain_ptr` atomic?

`retain_ptr` is only atomic in its reference count increment and decrements if the object it manages is itself atomic in its reference count operations.

§ 5.4. Does `retain_ptr` support allocators?

`retain_ptr` itself does not support allocators, however the object whose lifetime it extends can.

§ 5.5. Can `retain_ptr` be constexpr?

Possibly. However, I question the usefulness for a constexpr capable intrusive smart pointer, as most use cases are intended for non-constexpr capable interfaces such as incomplete types and polymorphic classes. Additionally, `retain_ptr` allows one to utilize value semantics on C and C++ APIs. If this is desired in a constexpr context, one can simply use constexpr values (i.e., reference counting is not a necessary aspect of constexpr)

§ 5.6. Does `retain_ptr` adopt or retain an object on construction?

The default action that `retain_ptr` takes on construction or reset is determined by the `traits_type` for the `retain_ptr`. If the traits type has a member named `default_action`, the `retain_ptr` will use that to delegate to the correct constructor. If there is no type alias member named `default_action`, the default operation is to *adopt* the object (i.e., it does not increment the reference count during its construction). The `default_action` type alias *must* be either `adopt_object_t` or `retain_object_t`.

§ 5.7. Why provide `retain_object_t` and `adopt_object_t`?

`retain_object_t` and `adopt_object_t` act as sentinel types to provide explicit requests to either *extend* or *adopt* an object when constructing or resetting a `retain_ptr`. This mostly comes into play when interacting with APIs that return a borrowed (rather than a new) reference to an object without increment its reference count.

Technically, an `enum class retain : bool { no, yes }` would be possible. However, this would be the first time such an API is placed into the standard library. Using a boolean parameter, as found in

`boost::intrusive_ptr` is unsatisfactory and does not help describe the operation the user is requesting.

The names of these sentinels are available for bikeshedding. Some other possible names for `retain_object_t` include:

- `retain_element_t`
- `extend_element_t`
- `retainobj_t`
- `extendobj_t`

While `adopt_object_t` names include:

- `borrow_object_t`
- `borrow_element_t`
- `borrowobj_t`
- `adopt_element_t`
- `adoptobj_t`

§ 5.8. Does `retain_ptr` overload the addressof operator?

Originally, this proposal suggested `retain_ptr` might in some small edge case require an overload for the addressof `operator &`. This was, with absolutely no surprise, contentious and asked to be removed. However, Microsoft's `ComPtr<T>` overloads the addressof operator for initializing it via C APIs (i.e., APIs which initialize a pointer to a pointer). Since then, JeanHyde Meneide's `out_ptr` proposal [\[P1132\]](#) was written and thus solves this slight issue.

§ 5.9. Can `retain_traits` store state?

No. Any important state regarding the object or how it is retained, can be stored in the object itself. For example, if the reference count needs to be external from the object, `std::shared_ptr` would be a better choice.

§ 5.10. Why not just wrap a `unique_ptr` with a custom deleter?

This is an extraordinary amount of code across many existing libraries that would not be guaranteed to have a homogenous interface. For example, using `retain_ptr` with an OpenCL context object (without checking for errors in both implementations) is as simple as:

```
struct context_traits {
    using pointer = cl_context;
    static void increment (pointer p) { clRetainContext(p); }
    static void decrement (pointer p) { clReleaseContext(p); }
};

struct context {
    using handle_type = retain_ptr<cl_context, context_traits>;
    using pointer = handle_type::pointer;
    context (pointer p, retain_object_t) : handle(p, retain) { }
    context (pointer p) : handle(p) { }
private:
    handle_type handle;
};
```

Using the `unique_ptr` approach requires more effort. In this case, it is twice as long to get the same functionality.

```

struct context_deleter {
    using pointer = cl_context;
    void increment (pointer p) const {
        if (p) { clRetainContext(p); } // retain_ptr checks for null for us
    }
    void operator () (pointer p) const { clReleaseContext(p); }
};

struct retain_object_t { };
constexpr retain_object_t retain { };

struct context {
    using handle_type = unique_ptr<cl_context, context_deleter>;
    using pointer = handle_type::pointer;

    context (pointer p, retain_object_t) :
        context(p)
    { handle.get_deleter().increment(handle.get()); }

    context (pointer p) : handle(p) { }

    context (context const& that) :
        handle(that.handle.get())
    { handle.get_deleter().increment(handle.get()) }

    context& operator = (context const& that) {
        context(that.handle.get(), retain).swap(*this);
        return *this;
    }

    void swap (context& that) noexcept { handle.swap(that); }

private:
    handle_type handle;
};

```

As we can see, using `retain_ptr` saves effort, allowing us in most cases to simply rely on the "rule of zero" for constructor management. It will also not confuse/terrify maintainers of code bases where objects construct a `unique_ptr` with the raw pointer of another (and *unique ownership* is not transferred).

§ 6. Examples

Some C APIs that would benefit from `retain_ptr<T>` are

- OpenCL
- Mono (Assembly and Image types)
- CPython
- ObjC Runtime
- Grand Central Dispatch

Inside the [\[implementation\]](#) repository is an extremely basic example of using `retain_ptr` with Python.

§ 6.1. OpenCL

As shown above, using `retain_ptr` with OpenCL is extremely simple.

```

struct context_traits {
    using pointer = cl_context;
    static void increment (pointer p) { clRetainContext(p); }
    static void decrement (pointer p) { clReleaseContext(p); }
};

struct context {
    using handle_type = retain_ptr<cl_context, context_traits>;
    using pointer = handle_type::pointer;
    context (pointer p, retain_object_t) : handle(p, retain) { }
    context (pointer p) : handle(p) { }
private:
    handle_type handle;
};

```

§ 6.2. ObjC Runtime

Additionally, while some additional work is needed to interact with the rest of the ObjC runtime, `retain_ptr` can be used to simulate ARC and remove its need entirely when writing ObjC. This means that one could, theoretically, write ObjC and ObjC++ capable code without having to actually write ObjC or ObjC++.

```

struct objc_traits {
    using pointer = CFTypeRef;
    static void increment (pointer p) { CFRetain(p); }
    static void decrement (pointer p) { CFRelease(p); }
};

```

§ 6.3. DirectX and COM

Because DirectX is a COM interface, it can also benefit from the use of `retain_ptr` by simply using the same common traits interface used for all COM objects. The following code is slightly adapted from Microsoft's `GpuResource` class in the DirectX Graphics Samples. The current form of the code uses the Microsoft provided `WRL::ComPtr`, however the point here is to show how `retain_ptr` can work as a drop-in replacement for this type.

```

struct com_traits {
    static void increment (IUnknown* ptr) { ptr->AddRef(); }
    static void decrement (IUnknown* ptr) { ptr->Release(); }
};

template <class T> using com_ptr = retain_ptr<T*, com_traits>;

struct GpuResource {

    friend class GraphicsContext;
    friend class CommandContext;
    friend class ComputeContext;

    GpuResource (ID3D12Resource* resource, D3D12_RESOURCE_STATES current) :
        resource(resource),
        usage_state(current)
    { }

    GpuResource () = default;

    ID3D12Resource* operator -> () noexcept { return this->resource.get(); }
    ID3D12Resource const* operator -> () const noexcept {
        return this->resource.get();
    }

protected:
    com_ptr<ID3D12Resource> resource { };
    D3D12_RESOURCE_STATES usage_state = D3D12_RESOURCE_STATE_COMMON;
    D3D12_RESOURCE_STATES transitioning_state = D3D12_RESOURCE_STATES(-1);
    D3D12_GPU_VIRTUAL_ADDRESS virtual_address = D3D12_GPU_VIRTUAL_ADDRESS_NULL;
    void* user_memory = nullptr;
};

```

§ 6.4. WebKit's RefPtr

As a small demonstration of replacing existing intrusive smart pointers with `retain_ptr` the author presents the following code from WebKit that uses the existing `RefPtr` type, followed by the same code expressed with `retain_ptr`. This is not meant to be a fully functioning code sample, but rather what the effects of a refactor to using `retain_ptr` might result in

```
RefPtr<Node> node = adoptRef(rawNodePointer);
```

```
auto node = retain_ptr<Node>(rawNodePointer, adopt_object);
```

§ 7. Technical Specification

A *retain pointer* is an object that extends the lifetime of another object (which in turn manages its own disposition) and manages that other object through a pointer. Specifically, a retain pointer is an object r that stores a pointer to a second object p and will cease to extend the lifetime of p when r is itself destroyed (e.g., when leaving a block scope). In this context, r is said to *retain* p , and p is said to be a *self disposing object*.

When p 's lifetime has reached its end, p will dispose of itself as it sees fit. The conditions regarding p 's lifetime is handled by some count c that p comprehends, but is otherwise not directly accessible to r .

The mechanism by which r retains and manages the lifetime of p is known as p 's associated *retainer*, a stateless object that provides mechanisms for r to increment, decrement, and (optionally) provide access to c . In this context, r is able to *increment* c , *decrement* c , or access the c of p .

Let the notation $r.p$ denote the pointer stored by r . Upon request, r can explicitly choose to increment c when $r.p$ is replaced.

Additionally, r can, upon request, *transfer ownership* to another retain pointer $r2$. Upon completion of such a transfer, the following postconditions hold:

- $r2.p$ is equal to the pre-transfer $r.p$, and
- $r.p$ is equal to `nullptr`

Furthermore, r can, upon request, *extend ownership* to another retain pointer $r2$. Upon completion of such an extension, the following postconditions hold:

- $r2.p$ is equal to $r.p$
- c has been incremented by 1

Each object of a type `U` instantiated from the `retain_ptr` template specified in this proposal has the lifetime extension semantics specified above of a retain pointer. In partial satisfaction of these semantics, each such `U` is `MoveConstructible`, `MoveAssignable`, `CopyConstructible` and `CopyAssignable`. The template parameter `T` of `retain_ptr` may be an incomplete type. (Note: The uses of `retain_ptr` include providing exception safety for self disposing objects, extending management of self disposing objects to a function, and returning self disposing objects from a function.)

```
class atomic_reference_count<T>;
class reference_count<T>;

class retain_object_t;
class adopt_object_t;

template <class T> struct retain_traits;

template <class T, class R = retain_traits<T>> class retain_ptr;

template <class T, class R>
void swap (retain_ptr<T, R>& x, retain_ptr<T, R>& y) noexcept;

template <class T, class R, class U>
retain_ptr<T, R> dynamic_pointer_cast (retain_ptr<U, R> const&) noexcept;

template <class T, class R, class U>
retain_ptr<T, R> static_pointer_cast (retain_ptr<U, R> const&) noexcept;

template <class T, class R, class U>
retain_ptr<T, R> const_pointer_cast (retain_ptr<U, R> const&) noexcept;

template <class T, class R, class U>
retain_ptr<T, R> reinterpret_pointer_cast (retain_ptr<U, R> const&) noexcept;

template <class T, class R, class S=R>
strong_ordering operator <=> (retain_ptr<T, R> const& x, retain_ptr<T, S> const& y) noexcept;

template <class T, class R>
strong_ordering operator <=> (retain_ptr<T, R> const& x, nullptr_t) noexcept;

template <class T, class R>
strong_ordering operator <=> (nullptr_t, retain_ptr<T, R> const& y) noexcept;
```

§ 7.1. `atomic_reference_count<T>` and `reference_count<T>`

`atomic_reference_count<T>` and `reference_count<T>` are mixin types, provided for user defined types that simply rely on `new` and `delete` to have their lifetime extended by `retain_ptr`. The template parameter `T` is intended to be the type deriving from `atomic_reference_count` or `reference_count` (a.k.a. the curiously repeating template pattern, CRTP).

```

template <class T>
struct atomic_reference_count {
    friend retain_traits<T>;
protected:
    atomic_reference_count () = default;
private:
    atomic<uint_least64_t> count { 1 }; // provided for exposition
};

template <class T>
struct reference_count {
    friend retain_traits<T>;
protected:
    reference_count () = default;
private:
    uint_least64_t count { 1 }; // provided for exposition
};

```

§ 7.2. retain_object_t and adopt_object_t

retain_object_t and adopt_object_t are sentinel types, with constexpr instances retain_object and adopt_object respectively.

```

namespace std {
    struct retain_object_t { constexpr retain_object_t () = default; };
    struct adopt_object_t { constexpr adopt_object_t () = default; };
    constexpr retain_object_t retain_object { };
    constexpr adopt_object_t adopt_object { };
}

```

§ 7.3. retain_traits<T>

The class template retain_traits serves the default traits object for the class template retain_ptr. Unless retain_traits is specialized for a specific type, the template parameter T must inherit from either atomic_reference_count<T> or reference_count. In the event that retain_traits is specialized for a type, the template parameter T may be an incomplete type.

```

namespace std {
    template <class T> struct retain_traits {
        static void increment (atomic_reference_count<T>*) noexcept;
        static void decrement (atomic_reference_count<T>*) noexcept;
        static long use_count (atomic_reference_count<T>*) noexcept;

        static void increment (reference_count<T>*) noexcept;
        static void decrement (reference_count<T>*) noexcept;
        static long use_count (reference_count<T>*) noexcept;
    };
}

```

```
static void increment (atomic_reference_count<T>* ptr) noexcept;
```

¹ Effects: Increments the internal reference count for ptr with memory_order::relaxed

² Postcondition: ptr->count has been incremented by 1.

```
static void decrement (atomic_reference_count<T>* ptr) noexcept;
```

¹ Effects: Decrements the internal reference count for ptr with memory_order::acq_rel. If the internal reference count of ptr reaches 0, it is disposed of via delete.

```
static long use_count (atomic_reference_count<T>* ptr) noexcept;
```

¹ Returns: The internal reference count for *ptr* with `memory_order::acquire`.

```
static void increment (reference_count<T>* ptr) noexcept;
```

¹ Effects: Increments the internal reference count for *ptr* by 1.

```
static void decrement (reference_count<T>* ptr) noexcept;
```

¹ Effects: Decrements the internal reference count for *ptr* by 1. If the count reaches 0, *ptr* is disposed of via `delete`.

```
static long use_count (reference_count<T>* ptr) noexcept;
```

¹ Returns: The reference count for *ptr*.

§ 7.4. retain_ptr<T>

The default type for the template parameter *R* is `retain_traits`. A client supplied template argument *R* shall be an object with non-member functions for which, given a *ptr* of type `retain_ptr<T, R>::pointer`, the expressions `R::increment(ptr)` and `R::decrement(ptr)` are valid and has the effect of retaining or disposing of the pointer as appropriate for that retainer.

If the *qualified-id* `R::pointer` is valid and denotes a type, then `retain_ptr<T, R>::pointer` shall be synonymous with `R::pointer`. Otherwise `retain_ptr<T, R>::pointer` shall be a synonym for `element_type*`. The type `retain_ptr<T, R>::pointer` shall satisfy the requirements of *NullablePointer*.

```

template <class T, class R=retain_traits<T>>
struct retain_ptr {
    using element_type = T;
    using traits_type = R;
    using pointer = /* see below */

    retain_ptr (pointer, retain_object_t);
    retain_ptr (pointer, adopt_object_t) noexcept;
    explicit retain_ptr (pointer);
    retain_ptr (nullptr_t) noexcept;

    retain_ptr (retain_ptr const&) noexcept;
    retain_ptr (retain_ptr&&) noexcept;
    retain_ptr () noexcept;
    ~retain_ptr ();

    retain_ptr& operator = (retain_ptr const&);
    retain_ptr& operator = (retain_ptr&&) noexcept;
    retain_ptr& operator = (nullptr_t) noexcept;

    void swap (retain_ptr&) noexcept;

    explicit operator pointer () const noexcept;
    explicit operator bool () const noexcept;

    element_type& operator * () const noexcept;
    pointer operator -> () const noexcept;
    pointer get () const noexcept;

    long use_count () const;

    [[nodiscard]] pointer release () noexcept;

    void reset (pointer, retain_object_t);
    void reset (pointer, adopt_object_t);
    void reset (pointer p = pointer { });
};

```

§ 7.4.1. `retain_ptr` constructors

```
retain_ptr (pointer p, retain_object_t);
```

Effects: Constructs a `retain_ptr` that retains `p`, initializing the stored pointer with `p`, and increments the reference count of `p` if `p != nullptr` by way of `traits_type::increment`.

Postconditions: `get() == p`

Remarks: If an exception is thrown during increment, this constructor will have no effect.

```
retain_ptr (pointer p, adopt_object_t) noexcept;
```

Effects: Constructs a `retain_ptr` that adopts `p`, initializing the stored pointer with `p`.

Postconditions: `get() == p`

Remarks: `p`'s reference count remains untouched.

```
explicit retain_ptr (pointer p);
```

Effects: Constructs a `retain_ptr` by delegating to another `retain_ptr` constructor via `traits_type::default_action`. If `traits_type::default_action` does not exist, `retain_ptr` is constructed as if by `retain_ptr(p, adopt_object_t)`.

Postconditions: `get() == p`

Remarks: If an exception is thrown, this constructor will have no effect.

```
retain_ptr () noexcept;
```

Effects: Constructs a `retain_ptr` object that retains nothing, value-initializing the stored pointer.

Postconditions: `get() == nullptr`

```
retain_ptr(retain_ptr const& r);
```

Effects: Constructs a `retain_ptr` by extending management from `r` to `*this`. The stored pointer's reference count is incremented.

Postconditions: `get() == r.get()`

Remarks: If an exception is thrown, this constructor will have no effect.

```
retain_ptr(retain_ptr&& r) noexcept;
```

Effects: Constructs a `retain_ptr` by transferring management from `r` to `*this`. The stored pointer's reference count remains untouched.

Postconditions: `get()` yields the value `r.get()` yielded before the construction.

§ 7.4.2. `retain_ptr` destructor

```
~retain_ptr();
```

Effects: If `get() == nullptr`, there are no effects. Otherwise, `traits_type::decrement(get())`.

§ 7.4.3. `retain_ptr` assignment

```
retain_ptr& operator = (retain_ptr const& r);
```

Effects: Extends ownership from `r` to `*this` as if by calling `reset(r.get(), retain)`. *Returns:* `*this`

```
retain_ptr& operator = (retain_ptr&& r) noexcept;
```

Effects: Transfers ownership from `r` to `*this` as if by calling `reset(r.release())`. *Returns:* `*this`

```
retain_ptr& operator = (nullptr_t) noexcept;
```

Effects: `reset()` Postconditions: `get() == nullptr` Returns: `*this`

§ 7.4.4. `retain_ptr` observers

```
element_type& operator * () const noexcept;
```

Requires: `get() != nullptr` Returns: `*get()`

```
pointer operator -> () const noexcept;
```

Requires: `get() != nullptr` Returns: `get()` Note: Use typically requires that `element_type` be a complete type.

```
pointer get () const noexcept;
```

Returns: The stored pointer

```
explicit operator pointer () const noexcept;
```

Returns: `get()`

```
explicit operator bool () const noexcept;
```

Returns: `get() != nullptr`

```
long use_count () const;
```

Returns: Value representing the current reference count of the current stored pointer. If `traits_type::use_count(get())` is not a valid expression, `-1` is returned. If `get() == nullptr`, `0` is returned.

Remarks: Unless otherwise specified, the value returned should be considered stale.

§ 7.4.5. `retain_ptr` modifiers

```
[[nodiscard]] pointer release () noexcept;
```

Postconditions: `get() == nullptr` Returns: The value `get()` had at the start of the call to `release()`.

```
void reset (pointer p, retain_object_t);
```

Effects: Assigns `p` to the stored pointer, and then if the old value of stored pointer `old_p`, was not equal to `nullptr`, calls `traits_type::decrement`. Then if `p` is not equal to `nullptr`, `traits_type::increment` is called. *Postconditions:* `get() == p`

```
void reset(pointer p, adopt_object_t);
```

Effects: Assigns `p` to the stored pointer, and then if the old value of stored pointer `old_p`, was not equal to `nullptr`, calls `traits_type::decrement`. *Postconditions:* `get() == p`

```
void reset (pointer p = pointer { })
```

Effects: Delegates assignment of `p` to the stored pointer via `reset(p, traits_type::default_action())`. *Postconditions:* `get() == p`

```
void swap (retain_ptr& r) noexcept;
```

Effects: Invokes `swap` on the stored pointers of `*this` and `r`.

§ 7.4.6. `retain_ptr` specialized algorithms

```
template <class T, class R>  
void swap (retain_ptr<T, R>& x, retain_ptr<T, R>& y) noexcept;
```

Effects: Calls `x.swap(y)`

```
template <class T, class R>  
auto operator <=> (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept = default;
```

Returns: `x.get() <=> y.get()`

```
template <class T, class R>  
auto operator <=> (retain_ptr<T, R> const&, nullptr_t) noexcept = default;
```

Returns: `x.get() <=> nullptr`

```
template <class T, class R>  
strong_ordering operator <=> (nullptr_t, retain_ptr<T, R> const& y) noexcept;
```

Returns: `nullptr <=> y.get()`

§ References

Informative References

[IMPLEMENTATION]

Isabella Muerte. [retain_ptr](#). URL: https://github.com/slurps-mad-rips/retain_ptr

[P1132]

JeanHeyd Meneide; Todor Buyukliev; Isabella Muerte. [out_ptr - a scalable output pointer abstraction](#).
URL: <https://wg21.link/p1132r0>